Weight Initialization Techniques | What Not to Do? | Deep Learning

➢ Introduction:

Every neural network begins with **random weights**.
But *how* you initialize them can make or break your model.
Bad initialization → exploding/vanishing gradients → poor convergence.
Good initialization → stable and fast learning.

➢ Why Weight Initialization is Important?

Weights determine:

- How activations flow through layers
- How gradients behave during backpropagation

If initialized incorrectly:

- Gradients may vanish (→ no learning)
- Gradients may explode (→ unstable learning)

Hence, initialization ensures:

1. Proper signal flow
2. Balanced gradient updates
3. Faster and more reliable convergence

➢ Case 1: All Weights = 0

If all weights are initialized to **zero**, every neuron receives the **same gradient** during backpropagation.
That means:

- All neurons learn the same features.
- Network fails to break symmetry.
- Model never learns meaningful patterns.

Rule #1: *Never initialize all weights to zero.*

➢ Case 2: Sigmoid Activation

Sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Derivative: $f'(x) = f(x)(1 - f(x))$

If weights are too **large**, the input to sigmoid gets pushed into the **saturated region** (very high or very low), where gradient ≈ 0.
This leads to the Vanishing Gradient Problem.

➢ Case 3: Very Small Random Weights

If weights are too **small**, the signal becomes extremely weak.
After multiple layers, outputs shrink toward **zero**, leading again to vanishing gradients.
Hence, both too large and too small initializations are bad.

➢ Case 3 (continued): Scaling Problem

Deep networks amplify or diminish signals exponentially as they pass through layers.
So, we need a strategy to **maintain variance** of activations and gradients across layers.
That's where *smart initialization methods* come in.

➢ ReLU Activation and Proper Initialization

ReLU works differently:

$$f(x) = max(0, x)$$

Only positive signals pass through.
Hence, half the neurons get **zero output** → if not initialized properly, training becomes unstable.

➢ Recommended Initializations

1. Xavier (Glorot) Initialization

Best for **Sigmoid / Tanh** activations.
Keeps variance of activations same across layers.

$$W \sim U\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right)$$

---

2. He Initialization (Kaiming Initialization)

Best for **ReLU** and its variants.
Compensates for ReLU's behavior (only half activations pass).

$$W \sim N\left(0, \frac{2}{n_{in}}\right)$$

---

3. LeCun Initialization

Best for **SELU** activation (self-normalizing networks).

$$W \sim N\left(0, \frac{1}{n_{in}}\right)$$

➢ Random Initialization (Large Values)

When initialized with very large values:

- Outputs and gradients blow up
- Network diverges
- Loss never stabilizes (Exploding Gradient Problem)

Rule #2: *Never initialize with large random values.*

➢ Outro / Key Takeaways

What to Do

- Use **Xavier** for Sigmoid/Tanh
- Use **He** for ReLU/Leaky ReLU
- Use **LeCun** for SELU
- Ensure variance of weights depends on number of inputs

What Not to Do

- Don't initialize all weights to zero
- Don't use large random numbers
- Don't ignore the activation function when choosing initialization

---

➢ Simple Definition

Weight Initialization determines the starting values of network parameters. Proper initialization ensures stable gradient flow and faster convergence, while poor initialization can lead to vanishing or exploding gradients.

➢ Mnemonic to Remember

Activation	Initialization	Hint
Sigmoid / Tanh	**Xavier**	"X" for cross symmetry
ReLU / Leaky ReLU	**He**	"He" helps ReLU stay healthy
SELU	**LeCun**	"LeCun loves SELU"
